

Kinds in Haskell and Their Analogues in F# and JavaScript

Jaanis Fehling — Università di Pisa

Abstract

Contents

1	Glossary of the Haskell Type System	2
2	Kinds in Haskell	2
2.1	Higher-kinded Types	3
3	Kinds and Types in JavaScript	3
4	Kinds and Types in F#	3
5	Conclusion	3
A	Supplementary Material	3

1 Glossary of the Haskell Type System

Before investigating kinds, we briefly explain the most important definitions in Haskell's type system and make example(s).

Type

A type classifies values and expressions; it describes what kind of data something is and which operations are valid on it.

```
Int, Bool, Char
[Int]
int -> Bool
```

Type Signature

A type signature is an explicit annotation of an expression's type.

```
not :: Bool -> Bool
```

Type Variable

A type variable is a placeholder for an unknown type, enabling polymorphism. Usually written *a*, *b*, *t*, etc.

```
id :: a -> a -- Here the type signature of "id" contains type variables
```

(Parametric) Polymorphism

Parametric polymorphism means code works uniformly for any type, without inspecting that type.

```
length :: [a] -> Int
```

2 Kinds in Haskell

Haskell offers besides a classical typesystem a "typesystem of a typesystem", also called *kinds* (i.e. kind of a type or type of a type). In GHCi, the interactive haskell interpreter, we can use the `:t` command to check the type of a previously declared variable:

```
ghci> x = 42 :: Int
ghci> :t x
x :: Int
```

Now, the type `Int` has the following kind, which we can check using the `:k` command:

```
ghci> :k Int
Int :: *
```

In contrast, we can check the kind of a type constructor such as `Maybe`:

```
ghci> :k Maybe
Maybe :: * -> *
```

`Maybe` is a type constructor, meaning it takes an additional type as an argument, which will then produce the kind:

```
ghci> :k Maybe Int
Maybe Int :: *
```

More complex kinds are also possible:

```
ghci> :k Either
Either :: * -> * -> *
ghci> :k (->)
(->) :: * -> * -> *
```

2.1 Higher-kinded Types

A higher-kinded type is a type constructor taking a type constructor as an argument:

```
ghci> :k Functor
Functor :: (* -> *) -> Constraint
```

Here `Constraint` is a typeclass constraint. We can test the `Functor` kind by supplying different types or type constructors:

```
ghci> :k Functor Maybe
Functor Maybe :: Constraint
ghci> :k Functor Int
```

```
<interactive>:1:9: error:
  - Expected kind '* -> *', but 'Int' has kind '*'
  - In the first argument of 'Functor', namely 'Int'
    In the type 'Functor Int'
```

3 Kinds and Types in JavaScript

4 Kinds and Types in F#

5 Conclusion

A Supplementary Material